

Lab 3 – ESP32 Setup and MicroPython

The purpose of this lab is to verify that your ESP32-C3 is working with firmware flashed, board joined to the umd-iot network, and MicroPython running code.

Step 1 is the ESP32 setup process covered in the lecture notes. If you already set up your ESP32 in class there is no need to repeat this step.

1. **Complete all ESP32 setup steps from the lecture notes.** SKIP THIS STEP if you already set up your ESP32 during lecture.

Flash MicroPython onto the board. Follow the firmware installation steps in the lecture notes: hold the BOOT button while plugging in the USB-C cable, then open Tools → Options → Interpreter, select MicroPython (ESP32) and the port whose name mentions USB JTAG, and click "Install or update MicroPython (esptool)". Use the ESP32-C3 family with the Wemos C3 mini variant, and leave "Erase all flash before installing" ticked. When it finishes, press the red STOP button and confirm that the shell shows the MicroPython banner and a >>> prompt.

Register your board on umd-iot. Copy `get_mac_address.py` to the board and run it to find your board's MAC address. Register that address at it.umd.edu/connect — you must be on campus WiFi or the campus VPN to reach the portal — and record the per-device password it gives you. That password belongs to this one board; your directory password will not work.

Put the course files on the board and connect. Download `main.py`, `wifi.py` and `config.py` from the course repository, edit `config.py` so that `UMD_IOT_PASSWORD` holds the password from step 2, and save all three to the MicroPython device without renaming them. Soft-reboot the board with ctrl-D and confirm that it reports a WiFi connection.

Install the I2C scanner with mip. At the >>> prompt, run these four lines. The scanner will report zero devices — nothing is wired to the bus yet — and that is the expected result. This step is worth doing because it only succeeds if the board has a real internet connection.

```
import mip
mip.install('github:ddevoe-
umd/enme441/micropython/03_ESP32_and_MicroPython/i2c_scanner.py')
import i2c_scanner
i2c_scanner.scan()
```

2. **Write lab3.py.** Using the provided lab3.py template file, write a function `get_ip()` that takes no arguments and returns (as a string) the IP address your board was given on umd-iot. Everything you need is in the provided `wifi.py` code: simply create a `network.WLAN` object so you can query the network interface, which should already be active when starting the ESP32, and call the `ifconfig()` method on the network interface object to get the IP address. You will need to slice the response from `ifconfig()` to pull out just the IP address string (in the same way as in `wifi.py`). Your function should return something like:

```
'123.45.6.789'
```

Run your code on the board with the `__main__` block left as-is.

To Submit:

Upload your finished file to Canvas. The file must be named `lab3.py`.

Grading (10 points):

- 3 `get_ip()` returns a valid IP address, as a string
- 3 `get_ip()` reads the address from the interface rather than returning one typed into the file
- 2 `get_ip()` returns the address itself, not the netmask, the gateway or the DNS server
- 1 `get_ip()` uses the station interface
- 1 running `lab3.py` prints the address without an error